

Full length article

Learnable faster kernel-PCA for nonlinear fault detection: Deep autoencoder-based realization

Zelin Ren ^{a,*}, Yuchen Jiang ^{b,*}, Xuebing Yang ^a, Yongqiang Tang ^a, Wensheng Zhang ^a

^a Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China

^b Control and Simulation Center, Harbin Institute of Technology, Harbin 150001, China

ARTICLE INFO

Keywords:

Kernel Principal Component Analysis (KPCA)

Fault detection

Process monitoring

Autoencoder

Data-driven

ABSTRACT

Kernel principal component analysis (KPCA) is a well-recognized nonlinear dimensionality reduction method that has been widely used in nonlinear fault detection tasks. As a kernel trick-based method, KPCA inherits two major problems. First, the form and the parameters of the kernel function are usually selected blindly, depending seriously on trial-and-error. As a result, there may be serious performance degradation in case of inappropriate selections. Second, at the online monitoring stage, KPCA has much computational burden and poor real-time performance, because the kernel method requires to leverage all the offline training data. In this work, to deal with the two drawbacks, a learnable faster realization of the conventional KPCA is proposed. The core idea is to parameterize all feasible kernel functions using the novel nonlinear DAE-FE (deep autoencoder based feature extraction) framework and propose DAE-PCA (deep autoencoder based principal component analysis) approach in detail. The proposed DAE-PCA method is proved to be equivalent to KPCA but has more advantage in terms of automatic searching of the most suitable nonlinear high-dimensional space according to the inputs, which helps to improve the accuracy of fault detection. Furthermore, the online computational efficiency improves by many times compared with the conventional KPCA. Finally, the Tennessee Eastman (TE) process benchmark and wastewater treatment plant (WWTP) benchmark are employed to illustrate the effectiveness of the proposed method, where the average fault detection rates of DAE-PCA are at least 0.27% and 4.69% higher than those of other methods, and its online computational efficiency is faster 90.48% and 24.57% times than that of KPCA respectively.

1. Introduction

Fault detection plays an important role in maintaining normal operating conditions and ensuring system safety, which has attracted extensive attention in recent years [1–3]. Efficient fault detection can greatly strengthen the system reliability and reduce unexpected economic loss [4]. With the increasing complexity of industrial processes, data-driven fault detection and process monitoring techniques take the dominant position in the field of fault detection and have become a research hotspot [5–11]. How to excavate the potential system information for different fault detection tasks from the massively available sensor data remains challenging.

Kernel principal component analysis (KPCA) is a typical and well-known data-driven method for dimensionality reduction based on kernel trick [12,13]. Owing to its good nonlinear feature extraction capability, KPCA and its extensions have been widely used for various fault detection and multivariate statistical process monitoring (MSPM) tasks

of industrial processes currently [14–16]. Zhang et al. [17] presented a fault detection frame of subspace reconstruction-based robust kernel principal component analysis model and it was employed to wind turbine systems. Simmini et al. [18] proposed a self-tuning KPCA method to detect the faults in chiller systems with a high accuracy. Deng et al. [19] integrated principal component analysis (PCA) and KPCA and presented a serial PCA that has the ability to better exploit the underlying process's structure in order to enhance fault diagnosis performance. Moreover, Deng et al. [20] also proposed an improved KPCA method by a deep architecture, called deep PCA (DePCA), which adopts the layerwise feature extraction strategy to obtain the multilevel data features for nonlinear process monitoring.

Although the kernel method has the ability to monitor nonlinear systems and detect faults by observing the fluctuations of features in high-dimensional space, it still has several problems and drawbacks.

* Corresponding authors.

E-mail addresses: rzl8816@126.com (Z. Ren), yc.jiang@hit.edu.cn (Y. Jiang), yangxuebing2013@ia.ac.cn (X. Yang), yongqiang.tang@ia.ac.cn (Y. Tang), zhangwenshengia@hotmail.com (W. Zhang).

<https://doi.org/10.1016/j.jii.2024.100622>

Received 27 June 2023; Received in revised form 2 January 2024; Accepted 28 April 2024

Available online 3 May 2024

2452-414X/© 2024 Published by Elsevier Inc.

First, the choice of kernel function has a great influence on the detection performance of the method. The forms of kernel function are multitudinous, and once a specific form of kernel function is determined, such as radial basis function (RBF) kernel, that means a subset of nonlinear mapping functions is determined. When its kernel parameters are chosen by offline testing (such as σ in RBF kernel), a specific nonlinear mapping is finally determined. It can be seen that this process has blindness, because the choice of kernel function totally depends on artificial selection and the kernel parameters are selected by trial and error. When there is no domain knowledge/expert experience, blind selection of kernel parameters will greatly limit the optimal selection of high-dimensional space. If the domain knowledge/expert experience is unsuitable for the working condition, it will also lead to unsatisfactory kernel parameters for fault detection. Thus, how to automatically search a proper kernel parameter is an intractable issue. To solve the problem, Tonin et al. [21] proposed a deep kernel PCA methodology (DKPCA) to extract multiple levels of the most informative components of the data. The approach can be regarded as a learnable deep KPCA to some extent by using neural network structure, but it does not specify the relationship between its nonlinear mapping function and the kernel function. Second, the online detection of the kernel method requires all offline samples. It causes a huge amount of calculation and has a serious influence when having demand on timely detection. In order to tackle the problem of low-efficiency online calculation existing in kernel methods, some researchers have tried to come up with some solutions in the past decade. Radhia Fezai et al. [22,23] developed an online reduced KPCA method and online reduced kernel GLRT technique to realize online and real time process monitoring by reducing the number of kernel functions. Ryan et al. [24] proposed a fast kernel transform (FKT) algorithm that achieves the combination of computational efficiency and broad applicability by leveraging a new general analytical expansion. Wang et al. [25] proposed a new idea of kernel sample equivalent replacement (KSER) method to display the kernel matrix as a quadratic form of the samples. Based on its benefits, Jiao et al. [26] presented a KPLS-KSER method by combining KPLS and KSER, which can effectively convert the kernel matrix calculation of new samples into linear operations in online fault detection, avoiding the exponential increase of computation time. However, none of the above methods simultaneously solve the problem from the selection of kernel parameters and online computational efficiency. Therefore, it is necessary to propose a new approximate nonlinear mapping method to replace current kernel methods.

To deal with the drawbacks of the existing KPCA-based methods, we consider reforming the DAE network to realize a learnable and faster KPCA by designing a network structure. Deep autoencoder (DAE) is a basic and popular data-driven deep learning method [27–30]. Due to its outstanding performance on powerful nonlinear representation for mass data, the potential of DAE has been demonstrated in industry. Recently, many researchers have presented a large number of variants, which have been prosperously applied to the fault detection and diagnosis area [8,30–33]. How to adopt DAE to efficiently serve various fault detection tasks, instead of kernel methods, has become the recent tendency.

In this article, we handle the aforementioned problems existing in kernel method by proposing a DAE-FE (deep autoencoder based feature extraction) framework, which includes an encoder module, a linear feature extraction (LFE) module, and a decoder module. We prove that the DAE-FE framework is essentially equivalent to a learnable and faster kernel method. Further, based on the DAE-FE framework, a DAE-PCA (deep autoencoder based principal component analysis) method is proposed to transform PCA into the corresponding nonlinear approach. The difference between KPCA and DAE-PCA is that the process that KPCA transforms a linear approach into the corresponding nonlinear version is composed of two stages. The first stage is to determine kernel function by manual selection. The second stage is to apply the

determined kernel function to convert the linear method into a nonlinear one. Therefore, for the kernel method, the setting of the kernel parameters is completely artificial and independent of the following linear method. The effect of nonlinearity depends entirely on artificial setting and has blindness. As for DAE-PCA method, it is an end-to-end learning process. When the network structure is determined, the encoder module and decoder module, as nonlinear modules, are learned together with the linear module in each back-propagation. The benefit of DAE-PCA is that the nonlinear modules serve the linear module and automatically search for a more suitable higher-dimensional space, which KPCA cannot achieve. In the detailed design of DAE-PCA, a PCA module is devised to achieve the function of PCA dimensionality reduction based on Cayley Transform [34]. The key contributions of our DAE-PCA can be summarized as follows.

- A DAE-FE framework is proposed for nonlinear fault detection. Based on the DAE-FE framework, a DAE-PCA method is further proposed, which is theoretically proven to be equivalent to a learnable KPCA. Compared with KPCA, the DAE-PCA method can automatically acquire the weight parameters of the network through training, unlike KPCA that selects kernel parameters blindly, depending seriously on trial-and-error.
- The proposed DAE-PCA method presents an alternative faster implementation of KPCA. In comparison with the conventional KPCA, the DAE-PCA method costs less time for online detection since its online detection time do not rely on the amount of training data, which greatly meets the requirement of real-time detection.
- To obtain a orthogonal load matrix, the idea of Cayley Transform is introduced to the design of the DAE-PCA method. Compared to the trick of introducing regularization term in the loss function, the orthogonal load matrix acquired by our proposed trick has a better orthogonal characteristic.

The rest of this article is organized as follows. In Section 2, KPCA and DAE are briefly illustrated. Section 3 first presents the DAE-FE framework. After that, the proposed implementation of DAE-PCA is presented for nonlinear fault detection. In Section 4, the effectiveness and feasibility of the proposed approach are demonstrated in the Tennessee Eastman (TE) process benchmark and wastewater treatment plant (WWTP) benchmark. Finally, Section 5 concludes this article.

2. Preliminaries

2.1. KPCA

Suppose that there are m sensors in an industrial system and each sensor collects N samples, the input variable matrix can be denoted as $\mathbf{X}$, i.e.,

$$\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]^T \in \mathbb{R}^{N \times m}.$$

As a typical linear self-supervised model, the traditional PCA [35] model achieves dimensionality reduction by the principle of capturing the maximum variance of the input data, and it can be formulated by minimizing the reconstruction error:

$$\begin{aligned} \min_{\mathbf{P}} & \|\mathbf{X} - \mathbf{XPP}^T\|_F^2 \\ \text{s.t. } & \mathbf{P}^T \mathbf{P} = \mathbf{I}_a, \end{aligned} \quad (1)$$

where $\mathbf{P} \in \mathbb{R}^{m \times a}$ is the orthogonal load matrix, a denotes the feature dimension after PCA operation, and $\|\cdot\|_F$ represents F -norm. Furthermore, we can obtain the dimensionality-reduction feature $\mathbf{T} \in \mathbb{R}^{N \times a}$, i.e., the score matrix, which can be calculated by

$$\mathbf{T} = [\mathbf{t}_1, \dots, \mathbf{t}_N]^T = \mathbf{XP} \in \mathbb{R}^{N \times a}, \quad (2)$$

KPCA is a nonlinear PCA method by introducing the kernel method. Kernel method aims to map the original process variables into a high-dimension reproducing kernel Hilbert space (RKHS) through a proper nonlinear mapping function, thus solving linearly inseparable problems [12]. Given a nonlinear mapping function $\phi(\cdot)$, it maps the original samples $\mathbf{x}_i$ ($i = 1, 2, \dots, N$) into a RKHS $\mathcal{H}$, i.e., $\mathbf{x}_i \in \mathbb{R}^m \rightarrow \phi(\mathbf{x}_i) \in \mathbb{R}^d$, where d is the dimension of $\mathcal{H}$. Thus, $\mathbf{X}$ is mapped into a nonlinear feature matrix Φ ,

$$\Phi = [\phi(\mathbf{x}_1), \phi(\mathbf{x}_2), \dots, \phi(\mathbf{x}_N)]^T \in \mathbb{R}^{N \times d}.$$

Notice that Φ needs to be centralized to be zero mean,

$$\bar{\Phi} = [\bar{\phi}(\mathbf{x}_1), \bar{\phi}(\mathbf{x}_2), \dots, \bar{\phi}(\mathbf{x}_N)]^T = \mathbf{H}_e \Phi. \quad (3)$$

where $\mathbf{H}_e = (\mathbf{I}_N - N^{-1} \mathbf{1}_N \mathbf{1}_N^T)$ is the centering matrix and $\mathbf{1}_N = [1, 1, \dots, 1]^T \in \mathbb{R}^N$.

Accordingly, the optimization problem of KPCA can be written as

$$\begin{aligned} \min_{\mathbf{P}} & \|\bar{\Phi} - \bar{\Phi} \mathbf{P} \mathbf{P}^T\|_F^2 \\ \text{s.t. } & \mathbf{P}^T \mathbf{P} = \mathbf{I}_a, \end{aligned} \quad (4)$$

where $\mathbf{P} \in \mathbb{R}^{d \times a}$ is the orthogonal load matrix. Then, the score matrix $\mathbf{T}$ is computed by $\mathbf{T} = \bar{\Phi} \mathbf{P}$.

When finishing model establishment, PCA or KPCA will map the original space into a principal component subspace ($\mathcal{PS}$) and a residual subspace ($\mathcal{RS}$), thereby achieving fault detection in these two subspaces.

2.2. DAE

The DAE can be trained to extract nonlinear system features by minimizing the reconstruction error of the input in an unsupervised self-learning manner. The main advantage of DAE is reconstructing the input data with fewer errors and effectively extracting the nonlinear feature. In DAE, the structure is made up of two parts: an encoder and a decoder. The encoder aims to extract the system features by nonlinear mapping of the input, while the decoder is used to reconstruct the input.

Let $\mathbf{X}$ and $\hat{\mathbf{X}} \in \mathbb{R}^{N \times m}$ be the input and output of DAE. The encoder and the decoder are respectively represented by nonlinear mappings $En(\cdot)$ and $De(\cdot)$, of which the nonlinearity is expressed by nonlinear activation functions. Formally, we have

$$\hat{\mathbf{X}} = De(En(\mathbf{X})). \quad (5)$$

The loss function for training DAE can be expressed by

$$\text{loss} = \mathcal{L}(\mathbf{X}, \hat{\mathbf{X}}) + \Omega, \quad (6)$$

where $\mathcal{L}(\cdot)$ is the reconstruction loss and Ω denotes regularization terms for specific problems or prior knowledge. The common choice of $\mathcal{L}(\cdot)$ is the sum of square errors,

$$\mathcal{L}(\mathbf{X}, \hat{\mathbf{X}}) = \|\mathbf{X} - \hat{\mathbf{X}}\|_F^2. \quad (7)$$

When the network has finished training, the extracted nonlinear system feature $\mathbf{T} \in \mathbb{R}^{N \times a}$ and the reconstruction error $\tilde{\mathbf{X}} \in \mathbb{R}^{N \times m}$ can be acquired by

$$\mathbf{T} = En(\mathbf{X}), \quad (8)$$

$$\tilde{\mathbf{X}} = \mathbf{X} - \hat{\mathbf{X}}. \quad (9)$$

For fault detection tasks, $\mathbf{T}$ and $\tilde{\mathbf{X}}$ can be utilized to monitor the $\mathcal{PS}$ and $\mathcal{RS}$, respectively.

3. Methodology

3.1. DAE-FE framework

The proposed DAE-FE framework is to transform linear feature extraction methods to their nonlinear versions using neural networks. The whole DAE-FE network structure is illustrated in Fig. 1, which consists of three modules: the encoder module, the LFE module, and the decoder module. The encoder and decoder modules aim to realize the transformation from the original input space $\mathcal{X}$ and RKHS $\mathcal{H}$ respectively. The purpose of the LFE module is to utilize a neural network to achieve the function of LFE, thereby obtaining nonlinear system features in $\mathcal{H}$. LFE is usually solved by matrix operation. In order to have the same form as other modules, it is necessarily transformed into the structure of neural network. In this way, the LFE module can combine the encoder module and the decoder module to jointly realize an end-to-end solution through back-propagation.

3.1.1. Encoder module

It utilizes $En(\cdot)$ to map the original input $\mathbf{X} \in \mathbb{R}^{N \times m}$ from the input domain $\mathcal{X}$ into a nonlinear feature space $\mathcal{H}$ to obtain the feature matrix $\Phi = En(\mathbf{X}) \in \mathbb{R}^{N \times d}$.

It will be proven in Proposition 1 that $En(\cdot)$ is in essence a learnable nonlinear function that can construct a kernel function. To describe the relationship between $En(\cdot)$ and kernel method, we need to give the following definition and lemma, of which the relevant proofs can be referred to the literature [36].

Definition 1. Given the N -dimensional input domain $\mathcal{X} \subset \mathbb{R}^N$, a function $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is called a kernel over $\mathcal{X}$ which is expressed by

$$\forall \mathbf{x}, \mathbf{x}' \in \mathcal{X}, K(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle = \phi(\mathbf{x})^T \phi(\mathbf{x}'),$$

where $\langle \cdot \rangle$ denotes inner product operation and $\phi(\cdot)$ is a nonlinear mapping function.

Lemma 1. Let kernel $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ be a continuous and symmetric function. Then, K is said to be a positive definite symmetric (PDS) kernel if for any $\{\mathbf{x}_1, \dots, \mathbf{x}_N\} \subseteq \mathcal{X}$, the Gram matrix $\mathbf{K} = [K(\mathbf{x}_i, \mathbf{x}_j)]_{i,j} \in \mathbb{R}^{N \times N}$ is symmetric positive semidefinite (SPSD).

Lemma 2. Let $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ be a PDS kernel. Then, there exists a Hilbert space $\mathcal{H}$ and a mapping $\phi(\cdot)$ from $\mathcal{X}$ to $\mathcal{H}$ such that:

$$\forall \mathbf{x}, \mathbf{x}' \in \mathcal{X}, K(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle.$$

Furthermore, $\mathcal{H}$ is called a RKHS associated with K if it has the following reproducing property:

$$\forall h \in \mathcal{H}, \forall \mathbf{x} \in \mathcal{X}, h(\mathbf{x}) = \langle h, K(\mathbf{x}, \cdot) \rangle.$$

Remark 1. Definition 1 gives the definition of the kernel function. The sufficient condition from Lemma 1 is a vital criterion to prove whether a function can realize to construct a corresponding kernel function. When Lemma 1 is satisfied, Lemma 2 tells that a RKHS associated with K must be exist.

The above preliminaries provide the theoretical basis to further prove why the function $En(\cdot)$ from the encoder module is equivalent to a learnable function whose inner product is a kernel function. Now, Proposition 1 is advanced as:

Proposition 1. The function $En(\cdot)$ from the encoder module is a nonlinear mapping whose inner product is a learnable kernel function. In addition, the space $\mathcal{H}$ associated with this learnable kernel is a RKHS.

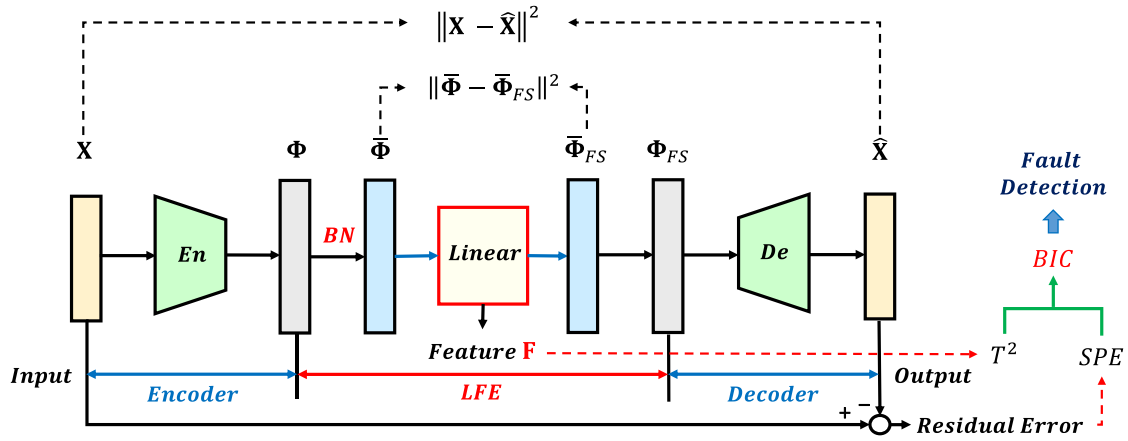


Fig. 1. The whole network structure of DAE-FE framework.

Proof. In the encoder module, the nonlinear activation functions are real-valued continuous functions and it only contain linear operators except for activation functions. Thus, as their composite functions, $En(\cdot)$ is a real-valued continuous function. According to Definition 1, a continuous and symmetric kernel K can be expressed by

$$K = \langle En(\cdot), En(\cdot) \rangle = En(\cdot)^T En(\cdot). \quad (10)$$

The input $\mathbf{X}$ is a matrix defined in the real domain. $\Phi = En(\mathbf{X})$ is also in the real domain. Thus, $\mathbf{K}$ is the Gram matrix with respect to K and $\mathbf{X}$ can be calculated by

$$\mathbf{K} = \Phi \Phi^T. \quad (11)$$

From (11), $\mathbf{K}$ is a real symmetric matrix. For any vector $\mathbf{x} \in \mathbb{R}^N$, it holds that

$$\mathbf{x}^T \mathbf{K} \mathbf{x} = \mathbf{x}^T \Phi \Phi^T \mathbf{x} = (\Phi^T \mathbf{x})^T (\Phi^T \mathbf{x}) \geq 0. \quad (12)$$

Thus, $\mathbf{K}$ is SPSD. According to Lemma 1, the kernel K is PDS kernel. Owing to Lemma 2, since K is a PSD kernel, the space $\mathcal{H}$ is a Hilbert space, and more specifically, a RKHS. $\square$

From Proposition 1, it can be seen that $En(\cdot)$ is in essence a nonlinear mapping function, of which the inner product is a learnable kernel function. Different neural network structures and parameters will correspond to a different $En(\cdot)$, and thus to a different kernel function. During training, the neural network executes back-propagation through loss function. When the training is over, a kernel function will be determined and can meet the requirements of the task.

3.1.2. LFE module

The major role of this module is to achieve linear feature extraction or dimensionality reduction for Φ .

In general, the input of LFE methods (such as PCA) needs to be standardized in each dimension, that is, the zero-mean and unit variance. Its main purpose is as follows: on one hand, the data of each dimension can be transformed to the same scale and pulled to the same baseline (origin of coordinates), so as to guarantee that each dimension has equal importance. On another hand, it is beneficial to the convergence of the gradient descent method.

To standardize Φ in $\mathcal{H}$ that is as the output of the encoder module, we introduce the Batch Normalization (BN) trick [37] into the encoder module. Two trainable parameters in BN trick require to be fixed as $\gamma = 1$ and $\beta = 0$. In specific, $\bar{\Phi}$ can be expressed as

$$\bar{\Phi} = BN(\Phi) = \frac{\Phi - \mu(\Phi)}{\sigma(\Phi)}, \quad (13)$$

where $\mu(\Phi)$ and $\sigma(\Phi)$ are the mean and standard deviation of Φ , respectively. Thus,

$$\frac{1}{N} \mathbf{1}_N^T \bar{\Phi} = \mathbf{0}. \quad (14)$$

It should be noted that the kernel method just executes centralization for Φ , because Φ in (3) is unknown, its variance cannot be processed to 1. Compared with the centralization in kernel method for Φ , our standardized treatment has better performance in data preprocessing.

The following network structure is designed in accordance with the given LFE method. The linear dimensionality reduction of $\bar{\Phi}$ is first performed to obtain the system feature $\mathbf{F}$. Then, $\bar{\Phi}$ in $\mathcal{H}$ is reconstructed as $\bar{\Phi}_{FS}$ by $\mathbf{F}$ under the constraint of loss function $loss_{\Phi}$, which is expressed as:

$$loss_{\Phi} = \|\bar{\Phi} - \bar{\Phi}_{FS}\|_F^2 + \lambda_{\Phi} \Omega_{\Phi}, \quad (15)$$

where λ_{Φ} is a non-negative hyper-parameter, and Ω_{Φ} is the regularization term in $loss_{\Phi}$.

Further, $\bar{\Phi}$ is decomposed into the following parts:

$$\bar{\Phi} = \bar{\Phi}_{FS} + \bar{\Phi}_{RS}, \quad (16)$$

$$\bar{\Phi}_{FS} = f_1(\mathbf{F}), \quad (17)$$

$$\mathbf{F} = f_2(\bar{\Phi}), \quad (18)$$

where $\bar{\Phi}_{RS}$ denotes the residual part of $\bar{\Phi}$, and $f_1(\cdot)$ and $f_2(\cdot)$ are both linear functions. The neural network from $\bar{\Phi}_{FS}$ to Φ_{FS} is a fully connected layer that is viewed as the inverse process of BN trick.

3.1.3. Decoder module

It is designed for the purpose of mapping the output Φ_{FS} of LFE module from $\mathcal{H}$ back to $\mathcal{X}$, and reconstruct the original input $\mathbf{X}$ under the constraint of the loss function $loss_{\mathbf{X}}$ as

$$loss_{\mathbf{X}} = \|\mathbf{X} - \hat{\mathbf{X}}\|_F^2. \quad (19)$$

Since the decoder network structure is almost symmetric with the encoder, it can be regarded as the inverse process of the encoder and effectively realizes the self-supervised learning for $\mathbf{X}$.

Through the above descriptions of DAE-FE framework, the entire loss function can be expressed as

$$loss = \lambda_1 loss_{\mathbf{X}} + \lambda_2 loss_{\Phi} + \lambda_3 \Omega, \quad (20)$$

where λ_i ($i = 1, 2, 3$) is a non-negative hyper-parameter, and Ω is the regularization term for a specific to a fault detection task. According to (20), all three modules of DAE-FE are jointly trained through back-propagation. After training DAE-FE network, we can use $\mathbf{F}$ to monitor FS and $\hat{\mathbf{X}}$ to monitor RS .

It is worth noticing that as the LFE module can be designed according to a specific LFE method, we can leverage DAE-FE as a learnable kernel method to convert any LFE method to the nonlinear version.

3.1.4. Kernel trick and DAE-FE

In this part, we describe the relationship and differences between the kernel trick and DAE-FE model.

The determination of kernel trick contains two aspects, one is the choice of kernel function and the other is the determination of its parameters. The determination of DAE-FE model also contains two aspects, one is the selection of neural network structure, and the other is the determination of network parameters. The DAE-FE model corresponds exactly to the kernel trick in these two aspects.

For the first aspect, the choice of kernel function in kernel trick determines the upper limit of nonlinear mapping ability. For example, the upper limit of nonlinear mapping ability of RBF kernel is theoretically higher than that of polynomial kernel. In DAE-FE model, the neural network structure determines the upper limit of nonlinear mapping ability by setting the number of network layers and activation functions. In general, the DAE-FE model does not need to specifically choose a nonlinear activation function. Most of the nonlinear activation functions have strong enough nonlinear ability to nonlinearly map the input into a linearly separable space when the number of network layers is sufficient. The higher the number of layers is, the stronger the nonlinear representation ability is. Usually, in the field of fault diagnosis, selecting Gaussian kernel and several layers of neural networks is enough to reach the upper limit required for nonlinear mapping. Therefore, the selection of DAE-FE network structure, such as the number of layers and the activation function, can meet the requirements as long as they are selected within a reasonable range.

More attention needs to be paid to the second aspect, namely the determination of the kernel and network parameters, since they determine at what level the kernel function and DAE-FE model can achieve within the upper limit. For the selection of kernel parameters, it relies on domain knowledge/expert experience or blind trial and error. In practical application, the selection of kernel parameters of different kernel functions does not follow the same rules, and kernel parameters with small numerical differences may lead to completely different fault detection results, which makes it difficult for the artificially specified kernel parameters to exert the nonlinear mapping ability of kernel function for fault detection tasks. Perhaps, the selected kernel parameters may be effective, but the effect is hardly satisfactory. Different from kernel parameters, the DAE-FE network parameters are acquired by backpropagation. Given initial values, the network can automatically learn the optimal parameters according to the input data, in order to obtain the nonlinear mapping ability that is most suitable for the current fault detection task. Compared with the kernel trick, the DAE-FE model has a glaring advantage that its network parameters have the ability to learn automatically with no blindness.

3.2. DAE-PCA method

In order to achieve the function of PCA, the matrix $\mathbf{P}$ learned by the neural network must be orthogonal. In other words, hard constraint is necessary to guarantee the orthogonality during the training process. We propose to solve this issue with the aid of Cayley Transform. Our DAE-PCA method is illustrated in Fig. 2.

3.2.1. PCA module

For this module, the optimization problem can be described like (4) of KPCA, i.e.,

$$\min_{\mathbf{P}} \|\bar{\Phi} - \bar{\Phi}\mathbf{P}\mathbf{P}^T\|_F^2 \quad (21)$$

s.t. $\mathbf{P}^T\mathbf{P} = \mathbf{I}_a$.

Note that the input of the PCA module is obtained by the encoder module, rather than kernel trick.

Let the loss function $loss_{\Phi, \text{PCA}}$ of PCA module be $loss_{\Phi}$ without Ω_{Φ} :

$$loss_{\Phi, \text{PCA}} = \|\bar{\Phi} - \bar{\Phi}_{FS}\|_F^2 = \|\bar{\Phi} - \bar{\Phi}\mathbf{P}\mathbf{P}^T\|_F^2. \quad (22)$$

Thus, in DAE-PCA, training the network to minimize $loss_{\Phi, \text{PCA}}$ is equivalent to $\min_{\mathbf{P}} \|\bar{\Phi} - \bar{\Phi}\mathbf{P}\mathbf{P}^T\|_F^2$. Hence, the system feature $\mathbf{T}$ is obtained by

$$\mathbf{T} = \bar{\Phi}\mathbf{P}. \quad (23)$$

At the same time, we need to restrict $\mathbf{P}$ to satisfy the constraint that $\mathbf{P}^T\mathbf{P} = \mathbf{I}_a$. For such purpose, we devise the neural network based on Cayley Transform to acquire $\mathbf{P}$. Cayley Transform was first established by Arthur Cayley, one proposition of which is given as follows [34].

Proposition 2. If $\mathbf{S}$ is a real antisymmetric matrix and $\mathbf{I}$ is the identity matrix, then $\mathbf{A} = (\mathbf{I} - \mathbf{S})(\mathbf{I} + \mathbf{S})^{-1}$ is an orthonormal matrix.

$\mathbf{P}$ can be directly obtained by the network structure as shown in the solid green border in Fig. 2, of which the input is fixed to the identity matrix $\mathbf{I}_{d \times d}$ and the output is an orthogonal matrix $\mathbf{P}$ that is acquired by the neural networks restricted based on Cayley Transform. Through training the entire DAE-PCA network with back-propagation mechanism, an optimal $\mathbf{P}$ suitable for the PCA module can be learned. The detailed descriptions for acquiring $\mathbf{P}$ in solid green border are in the following steps:

- (1) Fix the input to be identity matrix $\mathbf{I} \in \mathbb{R}^{d \times d}$ and initialize a square matrix $\mathbf{M}_0 \in \mathbb{R}^{d \times d}$. Thus, $\mathbf{M}_0$ is obtained by $\mathbf{M}_0 = \mathbf{M}_0\mathbf{I}$;
- (2) Get the upper triangular matrix $\mathbf{M}_1 \in \mathbb{R}^{d \times d}$ of $\mathbf{M}_0$;
- (3) Obtain the matrix $\mathbf{S} \in \mathbb{R}^{d \times d}$ by $\mathbf{S} = \mathbf{M}_1 - \mathbf{M}_1^T$;
- (4) According to Proposition 2, obtain an orthogonal square matrix $\mathbf{A} \in \mathbb{R}^{d \times d}$ by $\mathbf{A} = (\mathbf{I} - \mathbf{S})(\mathbf{I} + \mathbf{S})^{-1}$;
- (5) Take the first a orthogonal column vectors of $\mathbf{A}$ to get the orthogonal projection matrix $\mathbf{P} \in \mathbb{R}^{d \times a}$, where a is similar to the principle component number in PCA;
- (6) The most optimal projection matrix $\mathbf{P}$ for the PCA module will be acquired by back-propagation mechanism.

Notice that learning for matrix $\mathbf{P}$ is in essence to learn about matrix $\mathbf{M}_0$. Once $\mathbf{M}_0$ is determined, $\mathbf{P}$ is also determined.

The above trick is implemented by a hard constraint, rather than a soft constraint by adding any regularization term about orthogonality, i.e., $\Omega_{\Phi} = \|\mathbf{P}^T\mathbf{P} - \mathbf{I}_a\|_F^2$. Compared with the soft constraint, the advantage of our hard constraint is that its orthogonal performance is always maintained and is not affected by the design of the loss function.

Through the above analysis, the function $loss_{\text{DAE-PCA}}$ in our DAE-PCA is expressed as

$$loss_{\text{DAE-PCA}} = \lambda_1 loss_X + \lambda_2 loss_{\Phi, \text{PCA}} \quad (24)$$

$$= \lambda_1 \|\mathbf{X} - \hat{\mathbf{X}}\|_F^2 + \lambda_2 \|\bar{\Phi} - \bar{\Phi}_{FS}\|_F^2.$$

In addition, set $\lambda_1 = \frac{1}{N \times m}$ and $\lambda_2 = \frac{1}{N \times d}$ that aims to eliminate the effect of the number of matrix elements for each item of loss.

3.3. DAE-PCA based nonlinear fault detection

To detect whether some faults occur in the system adopting DAE-PCA method, we employ the two most commonly used statistics in fault diagnosis community to monitor $\mathcal{PS}$ and $\mathcal{RS}$ respectively, i.e., Hotelling's T^2 statistics and squared prediction error (SPE) statistics.

The flowchart of nonlinear fault detection based on DAE-PCA is summarized in Fig. 3. Given the training data matrix $\mathbf{X} \in \mathbb{R}^{N \times m}$, according to (23) and (9), we have the system feature matrix $\mathbf{T} \in \mathbb{R}^{N \times a}$ and residual matrix $\tilde{\mathbf{X}} \in \mathbb{R}^{N \times m}$. Set $\mathbf{x} \in \mathbb{R}^m$ to be a sample of $\mathbf{X}$, its Hotelling's T^2 statistics and SPE statistics can be defined as follows:

$$T^2(\mathbf{x}) = \mathbf{t}^T \mathbf{\Lambda}^{-1} \mathbf{t}, \quad (25)$$

$$SPE(\mathbf{x}) = \tilde{\mathbf{x}}^T \tilde{\mathbf{x}}, \quad (26)$$

where $\mathbf{t} \in \mathbb{R}^a$, $\tilde{\mathbf{x}} \in \mathbb{R}^m$ are the system feature vector and residual vector with respect to $\mathbf{x}$, and $\mathbf{\Lambda} = \mathbf{T}\mathbf{T}^T / (N - 1)$ is the covariance matrix of $\mathbf{t}$.

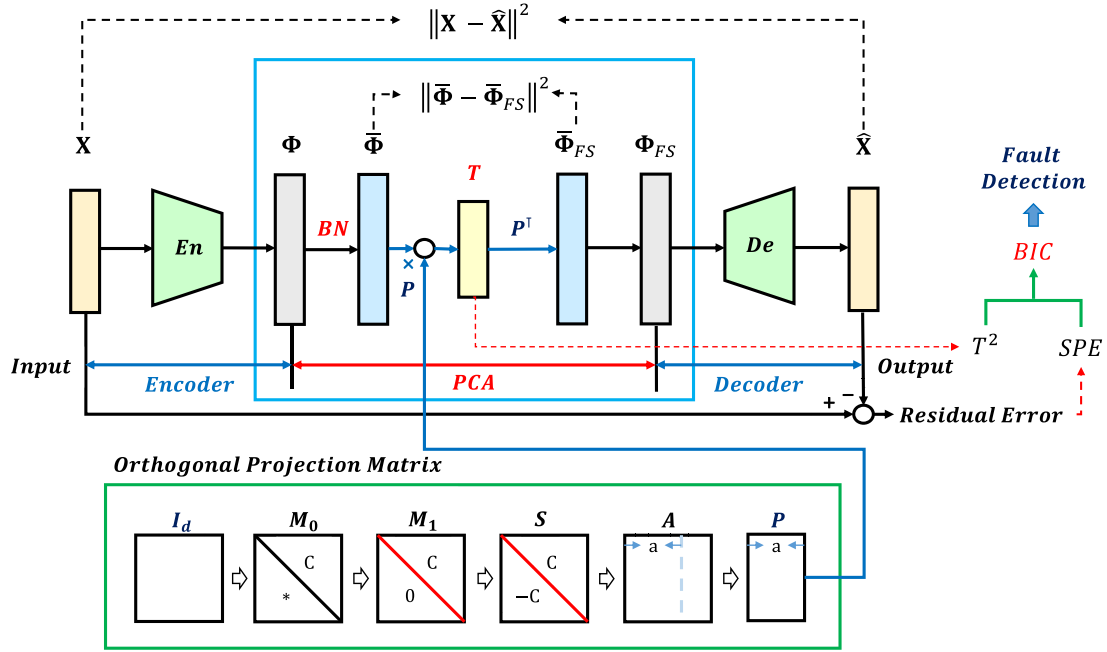


Fig. 2. The whole network structure of DAE-PCA, where the PCA module is shown with a solid blue border and the acquisition of $\mathbf{P}$ is in a solid green border.

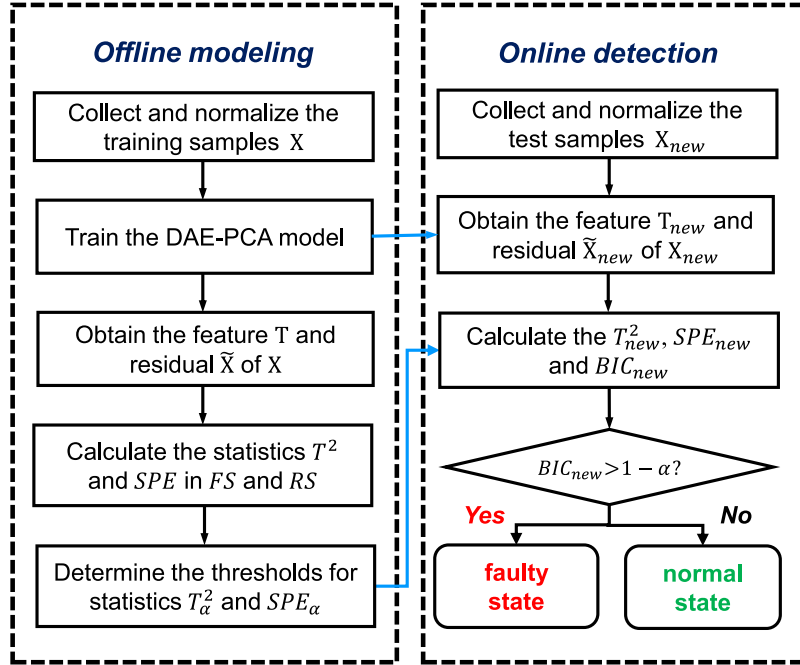


Fig. 3. The flowchart of nonlinear fault detection based on DAE-PCA.

We set the threshold of T^2 statistics and SPE statistics as J_{th,T^2} and $J_{th,SPE}$ respectively, which are calculated by applying T^2 and SPE to the kernel probability estimation (KDE) scheme under the given confidence limit α . The calculation method about KDE can be referred to [38].

For a test sample $\mathbf{x}_{new} \in \mathbb{R}^m$, let its system feature and residual be $\mathbf{t}_{new} \in \mathbb{R}^m$ and $\tilde{\mathbf{x}}_{new} \in \mathbb{R}^m$, respectively. Thus, T^2 statistics and SPE statistics of $\mathbf{x}_{new}$ are computed by

$$T^2(\mathbf{x}_{new}) = \mathbf{t}_{new}^T \Lambda^{-1} \mathbf{t}_{new}, \quad (27)$$

$$SPE(\mathbf{x}_{new}) = \tilde{\mathbf{x}}_{new}^T \tilde{\mathbf{x}}_{new}. \quad (28)$$

In order to provide a clear detection logic, the statistics T^2 and SPE are integrated into a statistics, Bayesian information criterion (BIC), according to the Bayesian inference [39,40]. To simplify the descriptions, these two statistics and their thresholds are represented by two sets, $S = \{T^2, SPE\}$ and $J_{th,S} = \{J_{th,T^2}, J_{th,SPE}\}$ respectively, since the following derivations are the same for them.

The fault posterior probability of S is calculated by

$$P_S(F|\mathbf{x}_{new}) = \frac{P_S(\mathbf{x}_{new}|F)P_S(F)}{P_S(\mathbf{x}_{new})}, \quad (29)$$

in which $P_S(\mathbf{x}_{new})$ is computed by

$$P_S(\mathbf{x}_{new}) = P_S(\mathbf{x}_{new}|N)P_S(N) + P_S(\mathbf{x}_{new}|F)P_S(F) \quad (30)$$

$$BIC(\mathbf{x}_{new}) = \frac{P_{T^2}(\mathbf{x}_{new}|F) P_{T^2}(F|\mathbf{x}_{new}) + P_{SPE}(\mathbf{x}_{new}|F) P_{SPE}(F|\mathbf{x}_{new})}{P_{T^2}(\mathbf{x}_{new}|F) + P_{SPE}(\mathbf{x}_{new}|F)}. \quad (33)$$

Box I.

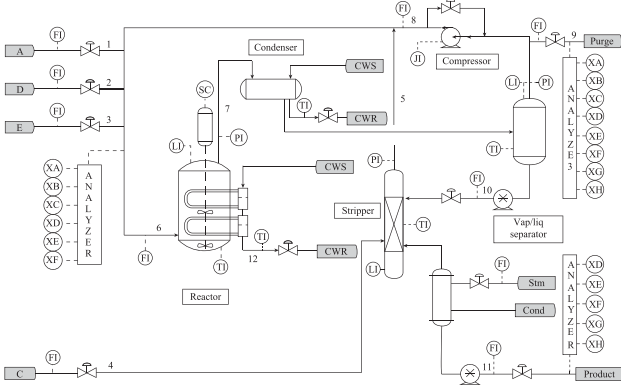


Fig. 4. The diagram of the TE process.

where N and F denote the normal and faulty state. The normal prior probability $P_S(N)$ equals to α and the faulty prior probability $P_S(F)$ is $1 - \alpha$. The normal and faulty conditional probability are defined as

$$\begin{aligned} P_S(\mathbf{x}_{new}|N) &= \exp\{-S/J_{th,S}\} \\ P_S(\mathbf{x}_{new}|F) &= \exp\{-J_{th,S}/S\}. \end{aligned} \quad (31)$$

Further, the statistics BIC can be obtained in (33), which is adopted to monitor the full space that is called FS . FS can be regarded as the space by combining PS with RS . Thus, the detection logic in FS is expressed as follows: (see the Eq. (33) in Box I).

$$\begin{cases} BIC(\mathbf{x}_{new}) \leq 1 - \alpha \Rightarrow \mathbf{x}_{new} \text{ is a fault-free sample} \\ BIC(\mathbf{x}_{new}) > 1 - \alpha \Rightarrow \mathbf{x}_{new} \text{ is a faulty sample} \end{cases} \quad (32)$$

4. Case study

To show the performance and the superiority of the proposed DAE-PCA, the TE process and WWTP are employed in our experiment. In this section, the proposed method will be compared with several comparative methods to comprehensively validate the performance in fault detection tasks.

4.1. Tennessee eastman process

4.1.1. Benchmark

The TE process is a real industrial benchmark which has been widely applied for the simulation and verification of process monitoring methods [41], and its diagram is displayed in Fig. 4. The TE process consists of two variable blocks: one is the XMV block composed by 11 manipulated variables and another is the XMEAS block that comprises 41 measured variables including 22 process variables and 19 analysis variables. Further information of the benchmark can be referred to [42] and the website.¹

Table 1

Fault types in the TE process.

No.	Description	Type
IDV(1)	A/C Feedratio, B composition constant (Stream 4)	Step
IDV(2)	B composition, A/C ratio constant (Stream 4)	Step
IDV(3)	D feed temperature (Stream 2)	Step
IDV(4)	Reactor cooling water inlet temperature	Step
IDV(5)	Condenser cooling water inlet temperature	Step
IDV(6)	A feed loss (Stream 1)	Step
IDV(7)	C header pressure loss (Stream 4)	Step
IDV(8)	A, B, C feed composition (Stream 4)	Random variation
IDV(9)	D feed temperature (Stream 2)	Random variation
IDV(10)	C feed temperature (Stream 4)	Random variation
IDV(11)	Reactor cooling water inlet temperature	Random variation
IDV(12)	Condenser cooling water inlet temperature	Random variation
IDV(13)	Reactor kinetics	Slow drift
IDV(14)	Reactor cooling water valve	Sticking
IDV(15)	Condenser cooling water valve	Sticking
IDV(16)	Unknown	Unknown
IDV(17)	Unknown	Unknown
IDV(18)	Unknown	Unknown
IDV(19)	Unknown	Unknown
IDV(20)	Unknown	Unknown
IDV(21)	Valve (Stream 4)	Constant position

4.1.2. Fault types

The description of the fault types is listed in Table 1. According to the difficulty level for faults to detect, they can be roughly divided into three categories as follows. The first includes the faults with large magnitudes: IDV(1), IDV(2), IDV(6)-IDV(8), IDV(12)-IDV(14), IDV(17), IDV(18), which are usually easily detectable. The second contains the faults: IDV(5), IDV(10), IDV(16), IDV(19)-IDV(21), which are relatively difficult to detect and the focus of detection. The third includes IDV(3), IDV(9) and IDV(15). They are incipient faults and usually excluded to be detected, because their mechanisms and modes are extremely complicated [43,44]. In our experiment, their detection results are still given, but they are not adopted for comparison.

4.1.3. Dataset

In this simulation, we choose 11 manipulated variables and 22 process variables to form the input matrix X . The training dataset and validation dataset are made up of 1168 normal samples and 292 normal samples respectively. For the test dataset, it consists of 21 faulty sets, each of which includes 960 samples, where the first 160 samples are fault-free and the rest are faulty.

4.1.4. Evaluation index

To evaluate the fault detection performance, two evaluation indexes, including the fault detection rate (FDR) and the false alarm rate (FAR) [32] are adopted in our experiments. A model is considered to be better than another model if its FDR is higher and FAR is lower.

4.1.5. Comparative method

Comparative methods include KPCA [45], KPCA-KSER [25], DAE [27]. Among these methods, KPCA is a classical kernel-based nonlinear PCA method. KPCA-KSER can be regarded as a kind of fast KPCA approximation method based on the recently proposed KSER technique, which is similar to KPLS-KSER [26]. DAE is a basic deep learning method for feature extraction.

¹ <http://depts.washington.edu/control/LARRY/TE/download.html>.

4.1.6. Model selection

Since both of DAE and DAE-PCA belong to the neural network approaches, a selection strategy should be given to determine which network model is more suitable for the fault detection task among all acquired neural network models. The specific strategy is provided as follow.

Given the initial values of neural network weights, a selection method to optimal model corresponding to these initial values is proposed through the following way: In each iteration of the network training, a network model will be acquired. When iterations reach $iter_{max}$, the $iter_{max}$ models will be obtained. Among these models, the model that has a minimum value of the loss function $loss_{DAE-PCA}$ for the validation set is selected as the optimal model corresponding to the current initial values, since it has the best generalization.

However, under different initial values of neural network weights, the network will learn different local optimal models which may have some differences to some extent in the performance of the fault detection task. Therefore, it is considered to select a more suitable model for the fault detection task among the models respectively obtained from multiple different initial values using the following steps: First, several sets of initial values are randomly generated by using the specified initialization trick. Then, each set of initial values are utilized to train the neural network to obtain their own models. After that, the model corresponding to each set of initial values is applied to the training dataset and the validation dataset (both are composed of normal samples) to obtain the values of the BIC statistics for all samples in these two datasets, and the values of these BIC statistics are summed. Finally, the model with the smallest total sum of the BIC statistics is finally selected as the optimal model for the fault detection task under the designed network structure, because the smaller the value of the BIC statistics, the more the samples of the training dataset and the validation dataset tend to be judged as normal samples.

4.1.7. Network setting

In neural network structure of DAE-PCA, we utilize ReLU as the activation function. The model parameters are $N = 1168$, $m = 33$, $d = 33$, $a = 30$, $\lambda_1 = (N \times m)^{-1} = 2.6 \times 10^{-5}$, $\lambda_2 = (N \times d)^{-1} = 2.6 \times 10^{-5}$. The number of nodes in each layer of the whole neural network is 33-33-30-33-33. The confidence level is $\alpha=0.99$. Besides, the Adam algorithm [46] is selected as the optimizer. The maximum $iter_{max}$ of training iteration ($iter$) is set to 2×10^4 and the learning rate (lr) is set piecewisely according to $iter$:

$$lr = 0.01 \times 0.7^{\lfloor iter/350 \rfloor} \quad (0 \leq iter \leq iter_{max}). \quad (34)$$

Here, the parameters of other methods are also given. For KPCA, it adopts RBF kernel and kernel parameter is $5\sqrt{330} (\approx 90.83)$, which is selected by referring to [45,47]. The number of the principal components is 25. The parameters in KPCA-KSER are same as those in KPCA, and the network structure of DAE is the same as that of DAE-PCA.

4.1.8. Implementation results

The FDRs for 21 faults using all methods are shown in Table 2. For faults in the first category, the FDRs of these four methods are almost over 90% in PS , RS and FS . It indicates that all comparative methods receive similar and satisfactory detection results. As for faults in the second category, DAE-PCA have better detection performance for full space than other methods, due to its higher values of FDRs. From the detection results of 18 kinds faults in the first and second categories, the proposed DAE-PCA has a higher FDRs than other methods in 15 kinds of faults, which indicates that DAE-PCA has better generalization ability in detecting different types of faults. In contrast, KPCA-KSER have relatively mediocre fault detection results. The reason for this result is that KPCA-KSER aims to greatly improve the online detection rate, but sacrifices a part of the accuracy of fault detection. In addition, in order to further reflect the average performance of various methods for different faults, Avg. is given as the average of statistics for all

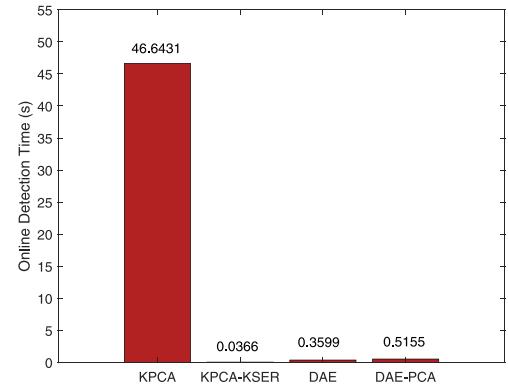


Fig. 5. The online detection time (second) of all methods. Here, the detection time refers to all time of 960 samples in a fault.

faults in Table 2. Because all methods fail to detect the third category of faults, the calculation of Avg. does not include these faults. The average values of FDRs for the BIC statistics in DAE-PCA method are the highest, which indicates that DAE-PCA method can produce a better mathematical expectation in FDRs.

Table 4 displays the FARs for 21 faults using all methods. From Table 4, DAE-PCA and KPCA-KSER has the lowest FARs and Avg. of FARs among all nonlinear methods in the full space. The results tell that DAE-PCA is not prone to false alarm for normal samples, thereby reducing manual useless checks.

To demonstrate that DAE-PCA method is a faster KPCA, the following experiment is given. In Table 3 and Fig. 5, we offer the offline training time and online detection time that are cost by all comparative methods respectively. It can be seen from Table 3 that the neural network methods generally spend more offline training time than the kernel methods, because the training process of the neural network requires a large number of iterative operations. However, in practical applications, there are usually adequate computing resources and time available to train fault detection models. Therefore, we usually do not pay much attention to the offline training time, since it is not an important consideration factor. But the online detection time is an important indicator of real-time performance, so it as a key factor for the quality of the detection method needs to be considered. Fig. 5 lists the online detection time t for all comparative methods. The online detection cost of KPCA is too heavy due to its kernel calculation that makes KPCA hard to apply in real systems. As a much fast KPCA approximation method, KPCA-KSER has the shortest online detection time among all comparative methods, because most operations in its algorithms are linear computation, which makes its online detection time equal to the linear method. DAE-based methods including DAE and DAE-PCA also cost less detection time. Although the detection time of DAE-based methods is slightly higher than that of KPCA-KSER, they can similarly meet the real-time requirements since they avoids the inner product calculation related to the kernel matrix.

In addition, by calculation, we get $\|P^T P - I_a\|_F^2 = 6.49 \times 10^{-15}$ from orthogonal projection matrix P . The result demonstrates that the hard constraint given by our PCA module has a quite precise orthogonal property.

Through the above analysis, we can know that DAE-PCA, as a learnable KPCA, has the same or even better detection results than KPCA and KPCA-KSER for different kinds of faults. Meanwhile, DAE-PCA, also as a faster KPCA, enables to better satisfy the real-time requirements than KPCA. Although the online detection speed of KPCA-KSER is slightly better than that of DAE-PCA, its fault detection accuracy is far worse than our proposed DAE-PCA, or even KPCA. Besides, DAE-PCA has better detection performance than DAE, which suggests the proposed DAE-FE nonlinear framework is much efficient.

Table 2

FDRs (%) of two subspaces PS and RS as well as the full space FS by KPCA, KPCA-KSER, DAE and DAE-PCA in the TE process. In addition, Avg. gives the average of the statistics for the first two categories of faults. The BIC statistics value in bold represents the best performance under the same fault (or Avg.) for all methods.

Fault No.	KPCA			KPCA-KSER			DAE			DAE-PCA		
	PS	RS	FS	PS	RS	FS	PS	RS	FS	PS	RS	FS
1	100	99.62	99.75	100	99.25	99.75	100	89.87	100	100	99.87	100
2	98.62	98.25	98.62	98.62	98	98.62	98.62	98	98.62	98.62	98	98.75
6	99.5	100	100	99.5	99.75	99.62	100	100	100	100	100	100
7	100	100	100	100	100	100	100	90.25	100	100	99.12	100
8	98	98.62	98.62	98	98.62	98.88	98.25	91	98.25	98.25	97.75	98.62
12	99	99.75	99.75	99	98.55	99.12	99.62	99.75	99.75	99.75	99.75	99.75
13	95.25	95	95.25	95.38	94.25	95.25	95.25	93.87	95.25	95.37	94	95.5
14	100	100	100	100	99.88	100	100	99.87	100	100	99.87	100
17	96.75	92.75	96.38	97	84.12	96.38	97.12	87.25	97.12	97.12	82.12	97.12
18	90.25	90.38	90.5	90.25	89.62	90	90.5	89.5	90.37	90.62	90.25	90.75
4	100	92.25	100	100	39.25	100	100	47.75	100	100	92	100
5	26.5	100	100	27	31	31.25	99.87	99.75	100	100	9.87	100
10	51.88	90.88	90.25	53.37	52.25	55.12	89	51.25	88.25	88.87	74	88
11	82.5	72.62	81.5	83.12	53.62	79.62	82.62	34.37	82	83.25	57.62	82.87
16	31.37	96.12	95.5	32.75	41.5	40	92.87	53.12	92.5	92.5	89.87	93.25
19	64.75	89.5	88	66.12	4.62	56.88	94.37	58.12	93.12	94.5	88.62	93.75
20	66.62	79.88	79.12	67.38	47.25	66.75	82.37	79.5	83.12	86.25	74.12	84.62
21	58.75	49.25	56.75	59.25	40.12	55.88	61	40.87	59.62	61	34.37	60.12
Avg.	81.1	91.38	92.78	81.49	70.64	81.28	93.41	77.89	93.22	93.67	87.29	93.51
3	4.5	13.75	10.38	4.75	9.38	7.62	8	1.75	5.88	7.5	4	7.38
9	3.7	9	6.88	4.12	6.5	5	5.75	2.25	6.5	5.5	2.88	5.5
15	10.12	18.38	17	11.38	15.62	14.5	13.25	3.38	12	13.25	5.5	12.63

Table 3

The offline training time (second) of all methods.

Time	KPCA	KPCA-KSER	DAE	DAE-PCA
t/s	5.9984	1.9081	215.357	186.562

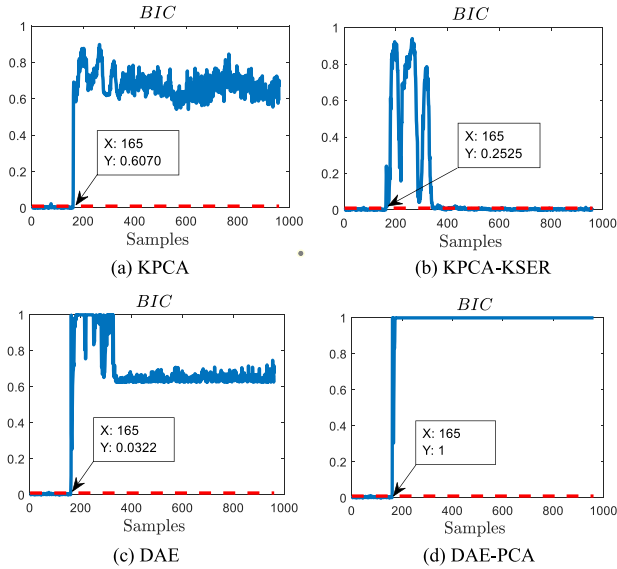


Fig. 6. Detection results of Fault IDV(5) in the TE process: (a) KPCA, (b) KPCA-KSER, (c) DAE and (d) DAE-PCA.

In order to observe the situation of the statistic BIC in FS of the sample in each moment, IDV(5) and IDV(19) are selected as the examples to illustrate the efficiency of the proposed DAE-PCA in detail. In this experiment, the detection delay is introduced as the evaluation index of whether the method can detect the fault in time. If five consecutive sample points indicate that the system is in the faulty state, a fault alarm will be raised at the fifth sample point.

IDV(5) is a step fault and the detection results for it is displayed in Fig. 6. The statistic BIC can approximate the probability that the sample is faulty. The greater the value of statistic BIC is, the greater

probability the sample has to be considered as a faulty sample. From Fig. 6, for faulty samples, KPCA-KPLS fails to detect IDV(5) since the values of its detection statistic BIC are lower than 50%. The statistic BIC values of KPCA and DAE fluctuate between 0.6 and 1, while those of DAE-PCA remain almost constant at 1. Regarding the samples in the normal state, the statistic BIC indexes of all methods are almost below the threshold 0.01. Besides, these four methods all give a fault alarm at the 165th sample without detection delays.

IDV(19) is a type-unknown fault. From Fig. 7, the detection results of all methods are provided. KPCA-KSER has relatively bad FDRs for statistics BIC in faulty samples, while the performance of KPCA are at an average level. In comparison with other methods, DAE-PCA shows outstanding results since its BIC indexes are almost far above the thresholds and approximate to 1 in most of faulty sample points. For normal samples, all methods are satisfactory and below the thresholds when the samples are free-fault. In detection delays, the delay time of DAE and DAE-PCA has one less sampling time than that of KPCA and KPCA-KSER.

Through the analysis of the above two faults, it implies that the proposed DAE-PCA has better separable property to handle the binary classification problem that whether the system is in a normal state or a faulty state, because DAE-PCA provides has a larger margin. Furthermore, it also strongly indicates that the kernel learned by DAE-PCA is more suitable for fault detection than that obtained by the predetermined kernel method and does not affect the accuracy of fault detection like KPCA-KSER.

4.2. Wastewater treatment plant

4.2.1. Benchmark

WWTP is a large-scale biochemical chemical system designed to treat wastewater into the water whose quality meets discharge standards. To facilitate the study of WWTP, a standard WWTP simulation model (Benchmark Simulation Model no. 1 (BSM1)) [48] has been developed to evaluate the performance of control strategies for activated sludge plants. WWTP is literally a complex nonlinear system, since the process of WWTP is affected by large perturbations which is caused by the uncertainty of wastewater inlet flow, pollutant load and wastewater composition. Hence, this benchmark has been widely used to assess the effectiveness of nonlinear fault detection methods for industrial processes.

Table 4

FARs (%) of two subspaces $\mathcal{P}\mathcal{S}$ and $\mathcal{R}\mathcal{S}$ as well as the full space $\mathcal{F}\mathcal{S}$ by KPCA, KPCA-KSER, DAE and DAE-PCA in the TE process. In addition, Avg. gives the average of the statistics for the first two categories of faults.

Fault	KPCA			KPCA-KSER			DAE			DAE-PCA		
No.	<i>PS</i>	<i>RS</i>	<i>FS</i>	<i>PS</i>	<i>RS</i>	<i>FS</i>	<i>PS</i>	<i>RS</i>	<i>FS</i>	<i>PS</i>	<i>RS</i>	<i>FS</i>
1	0.62	0.62	0.62	0.62	0.62	0	1.25	1.88	3.12	0.62	2.5	2.5
2	0.62	1.25	0.62	0.62	0	0.62	0.62	0.62	1.25	0.62	0.62	0.62
6	0	1.25	0.62	0	0	0	0	2.5	1.88	0	3.12	2.5
7	0	0.62	0	0.62	0	0	1.88	0	0.62	1.88	1.88	1.25
8	1.88	2.5	0.62	2.5	1.88	1.88	3.75	2.5	3.75	2.5	2.5	3.12
12	1.88	7.5	5	1.88	9.38	4.38	1.25	0.62	1.25	1.25	1.25	1.25
13	0	0	0	0.62	0.62	0	0	2.5	2.5	0.62	1.25	1.25
14	0.62	1.25	1.25	0.62	0	0	1.25	0.62	0.62	1.25	0.62	1.25
17	1.25	1.25	1.25	1.88	0.62	0.62	3.12	1.25	2.5	3.12	1.88	0.62
18	1.88	0.62	1.25	1.88	0	0.62	2.5	3.12	5	2.5	0.62	2.5
4	1.25	1.25	1.25	1.25	0.62	0.62	0.62	0	0.62	0.62	0	0.62
5	1.25	1.25	1.25	1.25	0.62	0.62	0.62	0	0.62	0.62	0	0.62
10	0.62	0.62	0.62	0.62	0.62	0.62	0.62	1.88	0.62	0.62	3.12	2.5
11	1.88	2.5	1.25	1.88	1.25	1.88	1.88	1.25	2.5	1.25	3.12	2.5
16	6.25	34.38	29.38	6.88	30	24.38	9.38	0	6.87	9.38	1.88	6.87
19	0	0.62	0.62	0	0	0	0.62	3.75	3.75	0.62	2.5	2.5
20	0	0.62	0.62	0.62	0	0	0	1.88	0.62	0	2.5	2.5
21	4.38	4.38	4.38	4.38	3.12	4.38	5	1.88	5.62	5	4.38	5.62
Avg.	1.35	3.47	2.81	1.56	2.74	2.26	1.91	1.46	2.43	1.8	1.87	2.26
3	0	7.5	4.38	0	5.62	1.88	3.12	1.88	2.5	4.38	2.5	3.75
9	8	21.25	20	9.38	12.5	10.62	11.25	3.12	10.63	11.88	5	11.25
15	0	1.88	0	0	0	0	1.88	1.25	1.88	1.88	1.88	2.5

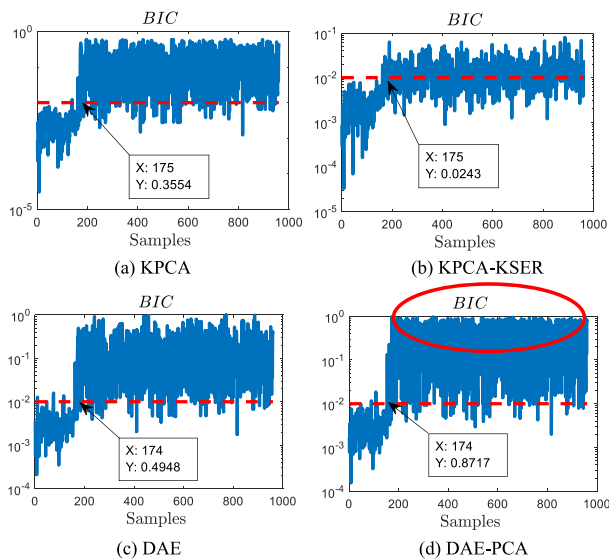


Fig. 7. Detection results of Fault IDV(19) in the TE process: (a) KPCA, (b) KPCA-KSER, (c) DAE and (d) DAE-PCA.

The BSM1 benchmark is composed of a five-compartment reactor and a secondary settler, and its structure flowchart is shown in Fig. 8. The BSM1 benchmark can be obtained from the website² and further information of the benchmark can be referred to its official website.³

4.2.2. Dataset and fault types

According to the literature [49], 18 variables (including 17 variables related to the biological phenomena and 1 variable related to flow rate of secondary settler inlet) are selected to form the input matrix $\mathbf{X}$ for the process monitoring, which are displayed in Table 5. In this experiment, the dynamic model is implemented with the dynamic dry weather influent data as the input to carry out a simulation under the

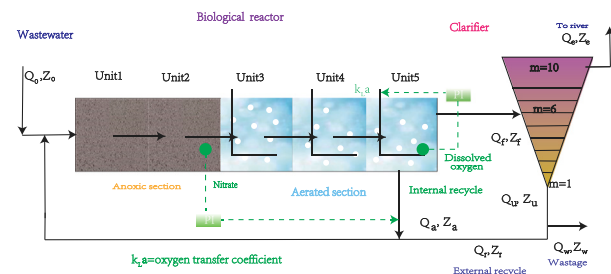


Fig. 8. Structure flowchart of the BSM1 benchmark plant in WWTP.

Table 5

Description and notation of the process variables chosen in the WWTP.

Description	Notation		
	Tank 3	Tank 4	Tank 5
Readily biodegradable substrate	$S_{S,3}$	$S_{S,4}$	$S_{S,5}$
Oxygen	$S_{O,3}$	$S_{O,4}$	$S_{O,5}$
Nitrate and nitrite nitrogen	$S_{NO,3}$	$S_{NO,4}$	$S_{NO,5}$
$NH_4^+ + NH_3$ nitrogen	$S_{NH,3}$	$S_{NH,4}$	$S_{NH,5}$
Alkalinity	$S_{ALK,3}$	$S_{ALK,4}$	$S_{ALK,5}$
	Settler inlet	Settler underflow	Plant exit
Flow rate	Q_f	—	—
Nitrate and nitrite nitrogen	—	$S_{NO,u}$	$S_{NO,e}$

standard MATLAB-Simulink environment. The data sample interval is 15 min. The training dataset and validation dataset are made up of 672 normal samples and 336 normal samples respectively. For the test dataset, it consists of four faulty sets, each of which includes 1344 samples, where the first 672 samples are fault-free and the rest are faulty. Four types of system faults are designed for this experiment, as shown in Table 6.

4.2.3. Network setting

In neural network structure of DAE-PCA, LeakyReLU is adopted as the activation function, of which the negative slope parameter is set to 0.001. The model parameters are $N = 672$, $m = 18$, $d = 18$, $a = 16$, $\lambda_1 = (N \times m)^{-1} = 8.3 \times 10^{-5}$, $\lambda_2 = (N \times d)^{-1} = 8.3 \times 10^{-5}$. The number of nodes in each layer of the whole neural network is 18-18-16-18-18. The

² <https://github.com/wwtmodels>.

³ <http://www.benchmarkwwtp.org>.

Table 6

Fault types in WWTP.

No.	Description
Fault 1	The maximum autotrophic growth rate (μ_A) in the whole process is increased from 0.5 d^{-1} to 0.8 d^{-1} .
Fault 2	The oxygen transfer rate in reactor number 3 ($K_{La,3}$) is decreased by half from 240 d^{-1} to 120 d^{-1} .
Fault 3	The decay of autotrophs kinetic parameter (b_A) in the whole process is doubled from 0.05 d^{-1} to 0.1 d^{-1} .
Fault 4	The maximum settling velocity of secondary settler (v_0') is decreased by half from 250 m d^{-1} to 125 m d^{-1} .

confidence level is $\alpha=0.99$. Besides, the Adam algorithm [46] is selected as the optimizer. The maximum $iter_{\max}$ of training iteration ($iter$) is set to 2×10^4 and the learning rate (lr) is set piecewisely according to $iter$:

$$lr = 0.01 \times 0.7^{\lfloor iter/300 \rfloor} \quad (0 \leq iter \leq iter_{\max}). \quad (35)$$

Here, the parameters of other methods are also given. For KPCA, it adopts RBF kernel and kernel parameter is $30\sqrt{5}$ (≈ 67.08). The number of the principal components is 5. The parameters in KPCA-KSER are same as those in KPCA, and the network structure of DAE is the same as that of DAE-PCA.

4.2.4. Implementation results

The FDRs for four faults using all methods are shown in Table 7. For Fault 1 and Fault 2, the FDRs for the BIC statistics in these four methods are all over 89% in $\mathcal{FS}$. It illustrates that they have similar and satisfactory abilities when detecting these two faults. As for Fault 3, KPCA-KSER fails to detect it when the fault occurs in WWTP. KPCA and DAE enables to detect the occurrence of Fault 3, but the detection results are less ideal due to their low FDRs. DAE-PCA receives most outstanding results among all methods and its FDR for the BIC statistics is 84.5% which is far higher than other methods. For this result, a proper analysis is conducted on why DAE-PCA has a more prominent fault detection rate than DAE. DAE directly adopts the features obtained from nonlinear mapping as system features to design the T^2 statistics. Different from DAE, DAE-PCA performs PCA decomposition on the features obtained from nonlinear mapping, which makes the features used to design the T^2 statistics orthogonal to each other. Thus, the features extracted from DAE-PCA can better characterize the state of the system than those from DAE. For the faults that are difficult to detect, such as Fault 3, DAE-PCA enables to better distinguish them from the normal state in comparison to DAE. While for faults that are relatively easy to detect, such as Fault 1 and Fault 2, both DAE-PCA and DAE have satisfactory fault detection results. When detecting Fault 4, DAE and DAE-PCA have well detection results since their BIC statistics have FDRs of 84%, which is higher than the 62% of KPCA and KPCA-KSER. From Avg. in Table 7, the average values of FDRs for the BIC statistics in DAE-PCA method are significantly higher than other comparative methods, which indicates that DAE-PCA method can produce a better mathematical expectation in FDRs. Table 8 shows the FARs for normal samples using all methods. From Table 8, DAE-PCA has the lowest FARs. The results indicate that false alarm for normal samples rarely happens when using DAE-PCA.

By combining the above results, it can be discovered that DAE-PCA, as a learnable KPCA, enables to find the nonlinear feature space that is more suitable for distinguishing the faulty state from the normal state, which makes DAE-PCA provide more accurate fault detection results than other comparative methods. Moreover, DAE-PCA also has a good ability to efficiently perform real-time online fault detection. As shown from Fig. 9, DAE-PCA has faster online computation rate compared with KPCA. Although offline training time is not considered as an evaluation factor in fault detection task, it is also given in the Table 9 for comprehensive understanding.

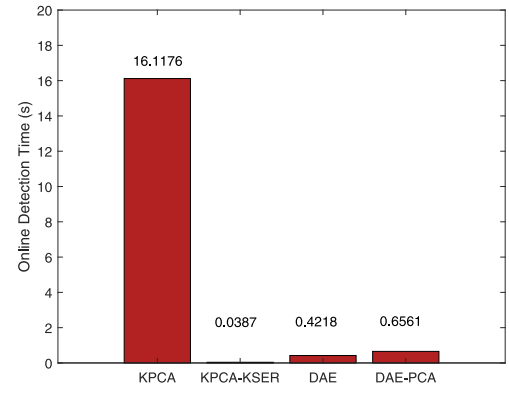


Fig. 9. The online detection time (second) of all methods. Here, the detection time refers to all time of 1344 samples in a fault.

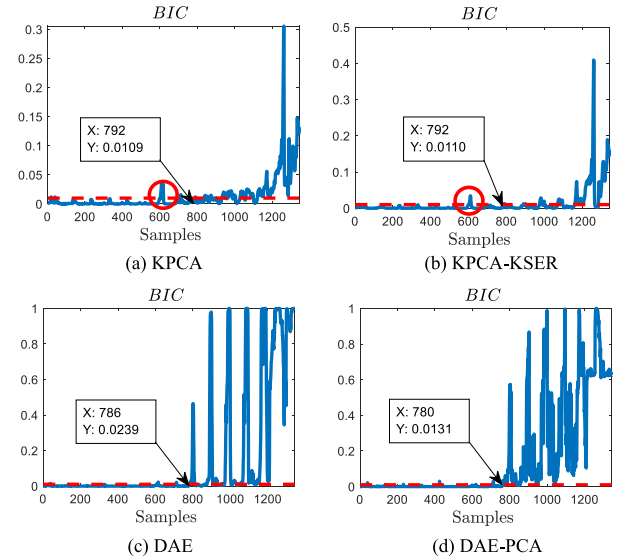


Fig. 10. Detection results of Fault 3 in the TE process: (a) KPCA, (b) KPCA-KSER, (c) DAE and (d) DAE-PCA.

Further, Fault 3 and Fault 4 are chosen to observe the situation of the statistics BIC in $\mathcal{FS}$ of the sample in each moment.

The detection results for Fault 3 is displayed in Fig. 10. When Fault 3 occurs in WWTP, most of the values of the BIC statistics of KPCA and KPCA-KSER fluctuate around the threshold. It demonstrates that these two methods both give low probabilities for the event that the fault samples are judged to be in the faulty state. In addition, KPCA gives false alarms between the 607th sample and the 613th sample, as well as in the 619th sample, while KPCA-KSER gives false alarms between the 604th sample and the 613th sample. These large numbers of false alarms seriously could mislead workers into believing that the normal system has existed the Fault 3, thereby fraying the bond of trust between workers and fault detection methods. DAE offers relatively mediocre detection results, because only a part of its BIC statistics values are far above the threshold, yet many of them below the threshold. For the proposed DAE-PCA, the values of its BIC statistics are larger than the threshold in a majority of fault samples. The results obviously reflect that the accuracy of DAE-PCA is higher than that of other methods in terms of detecting Fault 3. In the aspect of detection delay, DAE-PCA enables to raise a fault alarm for Fault 3 1.5 h earlier than DAE, and 3 h earlier than KPCA and KPCA-KSER, which leaves more time for workers to troubleshoot.

Table 7

FDRs (%) of two subspaces PS and RS as well as the full space FS by KPCA, KPCA-KSER, DAE and DAE-PCA in WWTP. In addition, Avg. gives the average of the statistics for the first two categories of faults. The BIC statistics value in bold represents the best performance under the same fault (or Avg.) for all methods.

Fault No.	KPCA			KPCA-KSER			DAE			DAE-PCA		
	PS	RS	FS	PS	RS	FS	PS	RS	FS	PS	RS	FS
Fault 1	90.61	90.46	97.76	90.46	14.75	89.72	95.53	100	100	90.91	100	100
Fault 2	89.87	100	100	89.87	21.16	89.57	100	100	100	99.85	100	100
Fault 3	34.87	53.35	56.63	34.72	0	33.83	48.88	67.51	66.17	48.58	86.44	84.5
Fault 4	64.68	28.32	61.85	63.93	2.83	62	65.57	83.9	83.9	66.62	84.8	84.35
Avg.	70.01	68.03	79.06	69.75	9.69	68.78	77.5	87.85	87.52	76.49	92.81	92.21

Table 8

FARs (%) of two subspaces PS and RS as well as the full space FS by KPCA, KPCA-KSER, DAE and DAE-PCA in WWTP. In addition, the BIC statistics value in bold represents the best performance for all methods.

Fault No.	KPCA			KPCA-KSER			DAE			DAE-PCA		
	PS	RS	FS	PS	RS	FS	PS	RS	FS	PS	RS	FS
Normal samples	3.27	1.78	2.97	3.12	1.78	2.23	1.04	0.89	1.49	1.04	1.04	1.19

Table 9

The offline training time (second) of all methods.

Time	KPCA	KPCA-KSER	DAE	DAE-PCA
t/s	1.048	0.5803	168.0466	175.0997

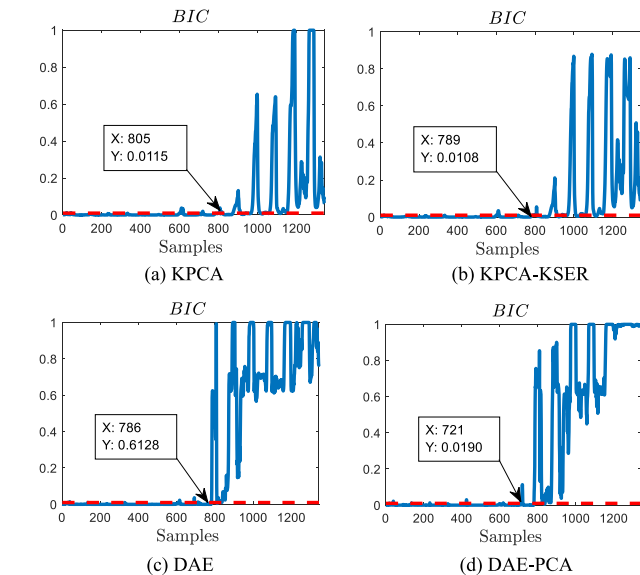


Fig. 11. Detection results of Fault 4 in the TE process: (a) KPCA, (b) KPCA-KSER, (c) DAE and (d) DAE-PCA.

Fig. 11 shows the relationship between the value of BIC statistics and the threshold in each sample when detecting Fault 4 by all comparative methods. It is obvious that when Fault 4 occurs in the system, the BIC statistics of DAE and DAE-PCA have much larger values than the threshold, so these two methods offer more reliable detection results by compared with KPCA and KPCA-KSER. In the terms of detection delay, DAE-PCA enables to raise a fault alarm for Fault 4 16.25 h earlier than KPCA, 17 h earlier than KPCA-KSER, and 21 h earlier than KPCA. Hence, DAE-PCA is more timely in detecting faults.

According to the above detection results for two kinds of fault in WWTP, DAE-PCA has higher fault detection accuracy by contrast with KPCA, KPCA-KSER and DAE. Moreover, DAE-PCA is more sensitive to monitor different kinds of faults, which is conducive to timely discovering the faults or failures. Therefore, DAE-PCA as a learnable and fast KPCA method is quite competitive in fault detection tasks.

5. Conclusion

In this paper, we present a DAE-PCA method based on a nonlinear DAE-FE framework that provides a way to transform a linear approach into the corresponding nonlinear version for fault detection tasks. The DAE-FE framework has been proven to equal to a corresponding learnable and faster kernel trick. In the case of fixed neural network structure, DAE-FE can automatically learn network weights that function similarly as the kernel parameters. Under this framework, we further design DAE-FE framework to put forward a DAE-PCA approach. Owing to Cayley Transform, the orthogonal projection matrix of PCA module in DAE-PCA method is guaranteed by hard constraints, which is implemented by the network structure. In comparison with KPCA, the proposed DAE-PCA has the ability to automatically learn an optimal parameter and has the faster computational efficiency. Through the experiment, the effectiveness of the proposed approach has been validated by detecting the faults in the TE process and WWTP. In future, the research will focus on how to modify DAE-PCA to be competent for the detection of incipient faults, such as IDV(3), IDV(9) and IDV(15) in the TE process. Besides, DAE-PCA will be further improved to achieve multi-level feature learning, similar to DePCA [20] and DKPCA [21].

CRediT authorship contribution statement

Zelin Ren: Conceptualization, Data curation, Formal analysis, Methodology, Software, Validation, Writing – original draft, Writing – review & editing. **Yuchen Jiang:** Investigation, Writing – review & editing, Visualization. **Xuebing Yang:** Visualization, Writing – review & editing. **Yongqiang Tang:** Writing – review & editing. **Wensheng Zhang:** Resources, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported in part by National Natural Science Foundation of China under Grant 62203143, and Heilongjiang Provincial Projects under Grants LJYXL2022-047, LBH-Z22130.

References

- [1] Z. Gao, C. Cecati, S.X. Ding, A survey of fault diagnosis and fault-tolerant techniques—part II: fault diagnosis with knowledge-based and hybrid/active approaches, *IEEE Trans. Ind. Electron.* 62 (6) (2015) 3768–3774.
- [2] Z. Huo, M. Martínez-García, Y. Zhang, R. Yan, L. Shu, Entropy measures in machine fault diagnosis: insights and applications, *IEEE Trans. Instrum. Meas.* 69 (6) (2020) 2607–2620.
- [3] K. Peng, K. Zhang, B. You, J. Dong, Z. Wang, A quality-based nonlinear fault diagnosis framework focusing on industrial multimode batch processes, *IEEE Trans. Ind. Electron.* 63 (4) (2016) 2615–2624.
- [4] Y. Jiang, S. Yin, J. Dong, O. Kaynak, A review on soft sensors for monitoring, control and optimization of industrial processes, *IEEE Sens. J.* 21 (11) (2021) 12868–12881.
- [5] E. Naderi, K. Khorasani, A data-driven approach to actuator and sensor fault detection, isolation and estimation in discrete-time linear systems, *Automatica* 85 (2017) 165–178.
- [6] I. Lomov, M. Lyubimov, I. Makarov, L.E. Zhukov, Fault detection in Tennessee eastman process with temporal deep learning models, *J. Ind. Inf. Integr.* 23 (2021) 100216.
- [7] H. Chen, Z. Liu, C. Alippi, B. Huang, D. Liu, Explainable intelligent fault diagnosis for nonlinear dynamic systems: From unsupervised to supervised learning, *IEEE Trans. Neural Netw. Learn. Syst.* (2022) <http://dx.doi.org/10.1109/TNNLS.2022.3201511>.
- [8] J. Liu, J. Wu, Y. Xie, W. Jie, P. Xu, Z. Tang, H. Yin, Toward robust process monitoring of complex process industries based on denoising sparse auto-encoder, *J. Ind. Inf. Integr.* 30 (2022) 100410.
- [9] H. Chen, L. Li, C. Shang, B. Huang, Fault detection for nonlinear dynamic systems with consideration of modeling errors: A data-driven approach, *IEEE Trans. Cybern.* (2022) <http://dx.doi.org/10.1109/TCYB.2022.3163301>.
- [10] Q. Jiang, S. Chen, X. Yan, M. Kano, B. Huang, Data-driven communication efficient distributed monitoring for multiunit industrial plant-wide processes, *IEEE Trans. Autom. Sci. Eng.* 19 (3) (2022) 1913–1923.
- [11] Q. Jiang, J. Jiang, W. Zhong, X. Yan, Optimized gaussian-process-based probabilistic latent variable modeling framework for distributed nonlinear process monitoring, *IEEE Trans. Syst. Man Cybern.: Syst.* 53 (5) (2023) 3187–3198.
- [12] B. Schölkopf, A.J. Smola, K.R. Müller, Nonlinear component analysis as a kernel eigenvalue problem, *Neural Comput.* 10 (5) (1998) 1299–1319.
- [13] Z. Ge, C. Yang, Z. Song, Improved kernel PCA-based monitoring approach for nonlinear processes, *Chem. Eng. Sci.* 64 (9) (2009) 2245–2255.
- [14] Q. Jiang, X. Yan, B. Huang, Review and perspectives of data-driven distributed monitoring for industrial plant-wide processes, *Ind. Eng. Chem. Res.* 58 (29) (2019) 12899–12912.
- [15] S. Yin, X. Li, H. Gao, O. Kaynak, Data-based techniques focused on modern industry: an overview, *IEEE Trans. Ind. Electron.* 62 (1) (2015) 657–667.
- [16] K. Zhang, K. Peng, R. Chu, J. Dong, Implementing multivariate statistics-based process monitoring: A comparison of basic data modeling approaches, *Neurocomputing* 290 (2018) 172–184.
- [17] K. Zhang, B. Tang, L. Deng, X. Yu, Fault detection of wind turbines by subspace reconstruction-based robust kernel principal component analysis, *IEEE Trans. Instrum. Meas.* 70 (2021).
- [18] F. Simmini, M. Rampazzo, F. Peterle, G.A. Susto, A. Beghi, A self-tuning KPCA-based approach to fault detection in chiller systems, *IEEE Trans. Control Syst. Technol.* 30 (4) (2022) 1359–1374.
- [19] X. Deng, X. Tian, S. Chen, C.J. Harris, Nonlinear process fault diagnosis based on serial principal component analysis, *IEEE Trans. Neural Netw. Learn. Syst.* 29 (3) (2018) 560–572.
- [20] X. Deng, X. Tian, S. Chen, C.J. Harris, Deep principal component analysis based on layerwise feature extraction and its application to nonlinear process monitoring, *IEEE Trans. Control Syst. Technol.* 27 (6) (2018) 2526–2540.
- [21] F. Tonin, Q. Tao, P. Patrinos, J.A.K. Suykens, Deep kernel principal component analysis for multi-level feature learning, 2023, arxiv preprint [arxiv:2302.11220](https://arxiv.org/abs/2302.11220).
- [22] R. Fezai, M. Mansouri, O. Taouali, M.F. Harkat, N. Bouguila, Online reduced kernel principal component analysis for process monitoring, *J. Process Control* 61 (2018) 1–11.
- [23] R. Fezai, M. Mansouri, M. Trabelsi, M. Hajji, H. Nounou, M. Nounou, Online reduced kernel GLRT technique for improved fault detection in photovoltaic systems, *Energy* 179 (2019) 1133–1154.
- [24] J.P. Ryan, S.E. Ament, C.P. Gomes, A. Damle, The fast kernel transform, in: *International Conference on Artificial Intelligence and Statistics*, PMLR, 2022, pp. 11669–11690.
- [25] G. Wang, J. Jiao, S. Yin, Efficient nonlinear fault diagnosis based on kernel sample equivalent replacement, *IEEE Trans. Ind. Inform.* 15 (5) (2019) 2682–2690.
- [26] J. Jiao, W. Zhen, G. Wang, Y. Wang, KPLS–KSER based approach for quality-related monitoring of nonlinear process, *ISA Trans.* 108 (2021) 144–153.
- [27] M. Sakurada, T. Yairi, Anomaly detection using autoencoders with nonlinear dimensionality reduction, in: *Proceedings of the MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis*, 2014, pp. 4–11.
- [28] I. Ahmed, T. Galoppo, X. Hu, Y. Ding, Graph regularized autoencoder and its application in unsupervised anomaly detection, *IEEE Trans. Pattern Anal. Mach. Intell.* 44 (8) (2022) 4110–4124.
- [29] A. Creswell, A.A. Bharath, Denoising adversarial autoencoders, *IEEE Trans. Neural Netw. Learn. Syst.* 30 (4) (2019) 968–984.
- [30] S. Chen, Q. Jiang, Distributed robust process monitoring based on optimized denoising autoencoder with reinforcement learning, *IEEE Trans. Instrum. Meas.* 71 (2022) 1–11.
- [31] P. Tang, K. Peng, J. Dong, Nonlinear quality-related fault detection using combined deep variational information bottleneck and variational autoencoder, *ISA Trans.* 114 (2021) 444–454.
- [32] K. Jang, S. Hong, M. Kim, J. Na, I. Moon, Adversarial autoencoder-based feature learning for fault detection in industrial processes, *IEEE Trans. Ind. Inform.* 18 (2) (2022) 827–834.
- [33] M. Rao, M.J. Zuo, Z. Tian, A speed normalized autoencoder for rotating machinery fault detection under varying speed conditions, *Mech. Syst. Signal Process.* 189 (2023).
- [34] A. Cayley, Sur quelques propriétés des déterminants gauches, *J. Reine Angew. Math.* 32 (1846) 119–123.
- [35] B.M. Wise, N. Ricker, D.V. eltkamp, B.R. Kowalski, A theoretical basis for the use of principal component models for monitoring multivariate processes, *Process Control Qual.* 1 (1) (1990) 41–51.
- [36] M. Mohri, A. Rostamizadeh, A. Talwalkar, *Foundations of Machine Learning*, MIT Press, 2018.
- [37] S. Ioffe, S.C. Szegedy, Batch normalization: Accelerating deep network training by reducing internal covariate shift, in: *International Conference on Machine Learning*, PMLR, 2015, pp. 448–456.
- [38] E. Parzen, On estimation of a probability density function and mode, *Ann. Math. Stat.* 33 (3) (1962) 1065–1076.
- [39] Z. Ge, M. Zhang, Z. Song, Nonlinear process monitoring based on linear subspace and Bayesian inference, *J. Process Control* 20 (5) (2010) 676–688.
- [40] J. Huang, X. Yan, Quality-driven principal component analysis combined with kernel least squares for multivariate statistical process monitoring, *IEEE Trans. Control Syst. Technol.* 27 (6) (2019) 2688–2695.
- [41] L.H. Chiang, E.L. Russell, R.D. Braatz, *Fault Detection and Diagnosis in Industrial Systems*, Springer Science & Business Media, 2000.
- [42] J.J. Downs, E.F. Vogel, A plant-wide industrial process control problem, *Comput. Chem. Eng.* 17 (3) (1993) 245–255.
- [43] Y. Wang, Z. Pan, X. Yuan, C. Yang, W. Gui, A novel deep learning based fault diagnosis approach for chemical process with extended deep belief network, *ISA Trans.* 96 (2020) 457–467.
- [44] Y. Zhang, Enhanced statistical analysis of nonlinear processes using KPCA, KICA and SVM, *Chem. Eng. Sci.* 64 (5) (2009) 801–811.
- [45] J.-M. Lee, C. Yoo, S.W. Choi, P.A. Vanrolleghem, I.-B. Lee, Nonlinear process monitoring using kernel principal component analysis, *Chem. Eng. Sci.* 59 (1) (2004) 223–234.
- [46] D.P. Kingma, B. Jimmy, Adam: A method for stochastic optimization, *Comput. Sci.* (2014).
- [47] L. Shang, K. Shi, C. Ma, A. Qiu, L. Hua, Fault detection and identification based on explicit polynomial mapping and combined statistic in nonlinear dynamic processes, *IEEE Access* 9 (2021) 149050–149066.
- [48] J. Alex, L. Benedetti, J.B. Copp, J.P. Steyer, Benchmark simulation model no. 1 (BSM1), in: *Report by the IWA Taskgroup on Benchmarking of Control Strategies for WWTPs*, 2008.
- [49] B. Wang, Z. Li, Z. Dai, N. Lawrence, X. Yan, Data-driven mode identification and unsupervised fault detection for nonlinear multimode processes, *IEEE Trans. Ind. Inform.* 16 (6) (2019) 3651–3661.